

Consignes :

- répondez à chaque problème **sur des feuilles séparées** et utilisez uniquement les feuilles de réponses qui vous sont données ;
 - indiquez votre nom, prénom et section sur chaque feuille de réponses ;
 - les notes de cours et personnelles **ne sont pas autorisées** ;
 - si ce n'est pas explicitement demandé dans l'énoncé :
 - **vous ne devez pas tester que les préconditions sont respectées** ;
 - **vos fonctions ne doivent pas avoir d'effet de bord** : les objets mutables passés en paramètres ne doivent pas être modifiés et rien ne doit être affiché à l'écran.
-

Problème 1 : quel est le mot de passe ?

Le but de cet exercice est d'implémenter une fonction permettant de découvrir un mot de passe. Le mot de passe à découvrir est composé uniquement des lettres de l'alphabet (au moins une lettre, et uniquement des minuscules). De plus, vous connaissez le nombre maximal de caractères qui peuvent le composer, mais pas le nombre exact. Par exemple : un mot de passe de longueur maximale 4 peut être composé de 1, 2, 3 ou 4 caractères minuscules.

Question :

Donnez un code python pour la fonction **itérative** `crack_password(pwlength)`. Le paramètre `pwlength` représente la taille maximale que le mot de passe peut atteindre. Pour trouver le mot de passe, vous utiliserez une *attaque par force brute*, qui consiste à tester, un à un, tous les mots de passe possibles jusqu'à trouver le bon.

Pour vous y aider, vous pouvez utiliser les deux fonctions booléennes suivantes (sans devoir les définir) :

- `is_letter_in_position(i, c)` qui retourne *True* si et seulement si la lettre `c` se trouve bien en position `i` du mot de passe à trouver.
- `perfect_match(mot)` qui retourne *True* si et seulement si `mot` correspond au mot de passe à trouver.

Vous ignorez tout de l'implémentation de ces méthodes, mais peu importe comment elles fonctionnent, elles vous renverront toujours le bon résultat.

Problème 2 : jeu de dominos

Un domino est une pièce rectangulaire sur laquelle figurent deux ensembles de points séparés par un trait. Le nombre de points de chaque côté du trait peut varier de 0 à 6. Dans cette question, nous considérons que les dominos sont orientés horizontalement. Ainsi, nous représenterons un domino par $[x|y]$ où x et y sont deux entiers compris entre 0 et 6 et où x (resp. y) est le nombre de points à gauche (resp. à droite) du trait. Notez que tout domino peut être *inversé* grâce à une rotation de 180 degrés : l'inverse de $[x|y]$ est $[y|x]$.

Le jeu des dominos consiste à étaler ses dominos sur une table en respectant une série de règles. L'une de ces règles précise qu'un domino ne peut être posé à côté d'un autre que si le nombre de points des cotés adjacents des deux dominos correspond. Dans le cadre de cette question, nous considérons un jeu de dominos dans une seule

dimension et dans lequel les dominos ne peuvent être ajoutés qu'à la droite d'une suite de dominos déjà placés. Ainsi, un domino A est dit *conciliable* avec un domino B s'il existe une façon de poser B à droite de A . On dit que A est *compatible* avec B si B est jouable à droite de A sans l'inverser.

Par exemple :

- $[3|1]$ est conciliable et compatible avec $[1|2]$;
- $[3|1]$ est conciliable mais pas compatible avec $[6|1]$;
- $[3|1]$ n'est ni conciliable ni compatible avec $[6|3]$.

Un domino $[x|y]$ sera représenté par un couple (x, y) en Python.

Questions :

- Implémentez une fonction `inverse(d)` qui retourne un nouveau domino correspondant à l'inversion du domino d passé en paramètre.
- Implémentez une fonction booléenne `compatible(d1, d2)` qui, étant donnés deux dominos $d1$ et $d2$, retourne vrai si les deux dominos sont compatibles, faux sinon.
- Implémentez une fonction booléenne `conciliable(d1, d2)` qui, étant donnés deux dominos $d1$ et $d2$, retourne vrai si les deux dominos sont conciliables, faux sinon.
- Implémentez une fonction `retirer(dominos, d)` qui, étant donnés un domino d et une liste de dominos `dominos`, retourne **une nouvelle liste** contenant les mêmes dominos que `dominos` moins une occurrence de d (inversé ou non). La liste `dominos` ne peut pas être modifiée par cette fonction. Par exemple, `retirer([(5, 3), (1, 2), (3, 5)], (3, 5))` peut retourner `[(1, 2), (3, 5)]`.
- Implémentez une fonction **récursive** `suite_dominos(dominos, debut)` qui, étant donnée une liste de dominos `dominos` et un domino initial `debut` passés en paramètre, retourne une liste de dominos dans laquelle les dominos forment une chaîne acceptable selon la règle donnée en début d'énoncé (chaque domino doit être compatible avec son successeur). Le domino initial ne peut être inversé. Si un tel ordre n'existe pas, la fonction retourne `None`. Par exemple, `suite_dominos([(1, 4), (1, 2), (5, 2), (3, 4)], (4, 5))` pourrait retourner le résultat suivant : `[(4, 5), (5, 2), (2, 1), (1, 4), (4, 3)]`.

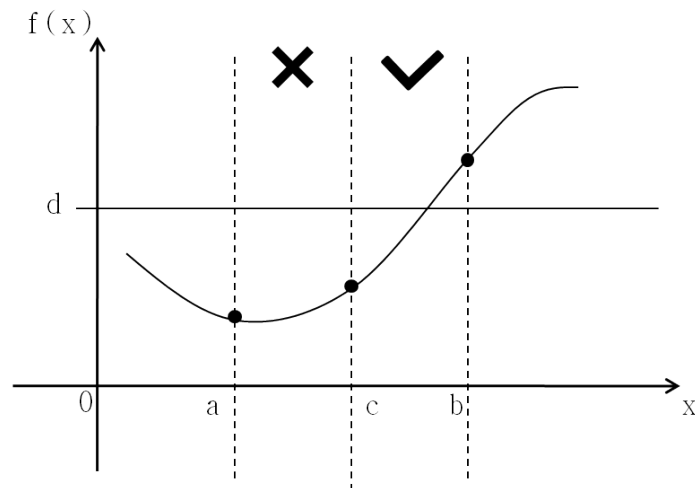
Notez que, bien que récursive, `suite_dominos` peut contenir des boucles.

Problème 3 : approximation d'une abscisse

Etant donné une fonction continue f et un réel d , le but de cet exercice est d'approximer une abscisse x telle que $f(x) = d$. Cette méthode commence par deux abscisses a et b représentant l'intervalle de recherche. Ces points sont choisis de telle sorte que $f(a) < d$ et $f(b) > d$ (*précondition*).

A chaque étape de cet algorithme, l'intervalle de recherche $[a, b]$ est découpé en deux intervalles de même longueur. Notons-les $[a, c]$ et $[c, b]$. L'abscisse c est alors une nouvelle approximation de l'abscisse recherchée. Ensuite, on réitère le processus avec l'un des deux intervalles vérifiant la precondition précitée. Notez que dans l'illustration suivante, l'intervalle marqué d'une croix ne respecte pas cette precondition.

L'algorithme s'arrête lorsque les approximations de deux étapes successives sont suffisamment proches (à un ϵ près).



Question

Donnez le code python d'une fonction **itérative** `find_abs(f, d, a, b, epsilon)` qui étant donnés une fonction `f`, un flottant `d`, un intervalle `[a, b]` et une précision `epsilon`, calcule une approximation d'une abscisse $x \in [a, b]$ telle que $f(x) = d$, suivant la méthode décrite précédemment. L'algorithme s'arrête lorsque la distance entre deux approximations, obtenues lors de deux étapes successives, est inférieure à `epsilon`.

Préconditions : `f` est continue dans l'intervalle `[a, b]`, $f(a) < d$ et $f(b) > d$.

Problème 4 : une élection étrange

Deux candidats se sont présentés à l'élection présidentielle de Weirland : le candidat *A* et le candidat *B*. Les bulletins de vote ont été rassemblés dans une urne¹ et les assesseurs s'apprêtent à dépouiller les résultats selon la méthode en vigueur dans leur pays, définie comme suit.

« Tant qu'il reste plus d'un seul bulletin de vote dans l'urne : prendre deux bulletins de vote au hasard dans l'urne ; si les deux bulletins tirés sont en faveur du même candidat, les jeter et ajouter un nouveau bulletin pour *A* (disponibles en quantité infinie) ; sinon (un bulletin *A* et un bulletin *B*), jeter le bulletin en faveur de *A* et remplacer le bulletin *B* dans l'urne. »

Durant la procédure, s'il ne reste qu'un seul bulletin, alors ce bulletin correspond au gagnant de l'élection. Les habitants de Weirland prétendent que ce type de dépouillement se termine toujours. De plus, et bien que la procédure est en partie aléatoire, certains affirment qu'il existe une règle déterministe permettant de connaître le candidat qui est déclaré vainqueur. La règle serait la suivante : « Si le nombre de bulletins pour *B* est pair, alors *A* est élu. Sinon, *B* est élu. » (★)

Cette procédure peut être simulée grâce à la fonction Python suivante qui prend une liste (Python) en paramètre ne contenant que des '*A*' ou des '*B*' (précondition) :

```
def depouillement(urne):
    while len(urne) > 1:
        print 'Urne au début de l\'itération:', urne,
        random.shuffle(urne)
        print ' -> ', urne
        votel = urne.pop()
```

1. On considère qu'un bulletin contient soit *A*, soit *B* et qu'il n'y a pas de bulletin blanc ou nul.

```

vote2 = urne.pop()
print "Votes retirés :", votel, vote2,
if votel == vote2:
    print '(identiques: A ajouté)'
    urne.append('A')
else:
    print '(différents: B ajouté)'
    urne.append('B')
print 'Urne à la fin de l\'itération:', urne, '\n', ('-' * 30)

```

La fonction `random.shuffle` permet de réordonner aléatoirement les éléments d'une liste passée en paramètre.

Par exemple, voici une exécution possible de `depouillement(urne)` si `urne` référence la liste `['A', 'A', 'B', 'B', 'A']` :

```

Urne au début de l'itération: ['A', 'A', 'B', 'B', 'A'] -> ['B', 'A', 'B', 'A', 'A']
Votes retirés : A A (identiques: A ajouté)
Urne à la fin de l'itération: ['B', 'A', 'B', 'A']
-----
Urne au début de l'itération: ['B', 'A', 'B', 'A'] -> ['A', 'B', 'B', 'A']
Votes retirés : A B (différents: B ajouté)
Urne à la fin de l'itération: ['A', 'B', 'B']
-----
Urne au début de l'itération: ['A', 'B', 'B'] -> ['A', 'B', 'B']
Votes retirés : B B (identiques: A ajouté)
Urne à la fin de l'itération: ['A', 'A']
-----
Urne au début de l'itération: ['A', 'A'] -> ['A', 'A']
Votes retirés : A A (identiques: A ajouté)
Urne à la fin de l'itération: ['A']

```

Selon cette procédure, *A* serait donc déclaré vainqueur.

Questions

On suppose que le paramètre `urne` référence une liste non vide ne contenant que des 'A' et des 'B'.

- Prouvez que la fonction `depouillement` se termine toujours.
- Si la liste `urne` contient initialement x 'B' et si l'on note par y le nombre de 'B' courant dans la liste (entre 2 itérations de la boucle), prouvez que l'assertion

$$S \equiv (x \text{ et } y \text{ sont pairs}) \text{ ou } (x \text{ et } y \text{ sont impairs})$$

est un invariant de la boucle de la fonction `depouillement`.

- Selon vous, la règle ($\star$), énoncée plus haut, est-elle toujours vraie ? (Justifiez, en utilisant les résultats prouvés aux points précédents si nécessaire.)